Backend Design Document

AI-Powered Smart Recruitment Platform

1. Overview

The backend is a production-ready REST API built with FastAPI following Clean Architecture. The system separates Presentation, Application, Domain, and Infrastructure layers. Business logic resides only in services while repositories encapsulate database access. The backend serves a React frontend through versioned APIs (/api/v1).

2. Technology Stack

Python 3.12+, FastAPI, SQLAlchemy 2.0, Alembic, SQLite, JWT, bcrypt, Pydantic v2, FastAPI BackgroundTasks, Google Gemini (via abstraction), Docker, Docker Compose, Nginx, Uvicorn.

3. Architecture Principles

SOLID, Clean Architecture, Repository Pattern, Service Layer, Unit of Work, Dependency Injection, centralized configuration, reusable utilities, strict typing, and environment-based configuration.

4. Project Structure

Modules: api, core, config, database, models, schemas, repositories, services, dependencies, security, middleware, ai, background, storage, logger, validators, utils, constants, enums, tests, migrations.

5. Authentication & Security

JWT access/refresh tokens, bcrypt password hashing, RBAC (Admin, Employer, Applicant), global exception handler, secure file validation, CORS, trusted hosts, audit logging, environment variables, standardized error responses.

6. Database Design

SQLite with SQLAlchemy ORM and Alembic migrations. Every entity contains audit fields (created_at, updated_at). Repositories isolate persistence from business logic.

7. Core Business Modules

Authentication, Employer (company and job management), Applicant (profiles, applications), Resume Management, AI Services, Search, Dashboard, and Admin. APIs support CRUD, pagination, filtering, sorting, and searching.

8. Resume Processing Pipeline

Upload PDF/DOC/DOCX → validate → store locally → BackgroundTasks extract text → parse candidate information → invoke Gemini provider → generate summary, skill extraction, semantic score, hiring recommendation → persist results.

9. AI Layer

Use an LLMProvider interface with a GeminiProvider implementation. Future providers can be added without modifying business logic. AI configuration is managed through settings and prompt templates.

10. API Standards

REST endpoints under /api/v1 with consistent request/response schemas. Standard response: success, message, data. Validation through Pydantic. Global exception middleware returns structured errors. Health and readiness endpoints included.

11. Logging

Centralized logger imported by all modules. Log format includes timestamp, level, filename, function, line number, and message. Supports console, rotating file logs, and configurable log levels.

12. Background Processing

FastAPI BackgroundTasks perform resume parsing and AI processing asynchronously to keep API responses responsive. Background services are isolated for future migration to dedicated task queues.

13. Deployment

Production Dockerfile, Docker Compose, Nginx reverse proxy, Uvicorn ASGI server, .env configuration, health checks, and readiness endpoints. Designed for container deployment on common cloud platforms.

14. Testing & Extensibility

Use unittest for repository, service, authentication, and API testing. The architecture supports future PostgreSQL migration, additional LLM providers, notifications, vector databases, semantic search, analytics, and microservices without major refactoring.

15. Development Standards

Follow PEP 8, type hints, minimal comments, reusable components, list comprehensions where appropriate, dependency injection, no duplicated business logic, and clear module boundaries.